Assignment 3

Name: Muhammad Awais

Question 1:

Data Warehouse, Data Lake, Data Lakehouse

Solution:

Data Warehouse:

Data warehouse is a centralized storage system where Data Engineers store cleaned, and structured data, that is fetched from various sources online. The data is already ready for analysis and further work by data analysts/engineers alike. It uses a schema on write approach, which means that the data being fetched follows some fixed schema as it's being written in the storage.

Data Lake:

Data Lake is also a centralized repository but it can hold large amounts of structured, unstructured and semi-structured data. The data does not follow any schema unlike in Data warehouse, so it's up to us to make use of the data stored, according to different domains.

Data Lakehouse:

Data Lakehouse is a hybrid style repository, taking the best of both worlds of data lake house and data warehouse, but this time your data can follow a nice structure and schema, just like in data warehouse. This would make sure that people from diverse backgrounds can use the data stored.

Reference: https://www.ibm.com/think/topics/data-warehouse-vs-data-lake-vs-data-lakehouse?utm

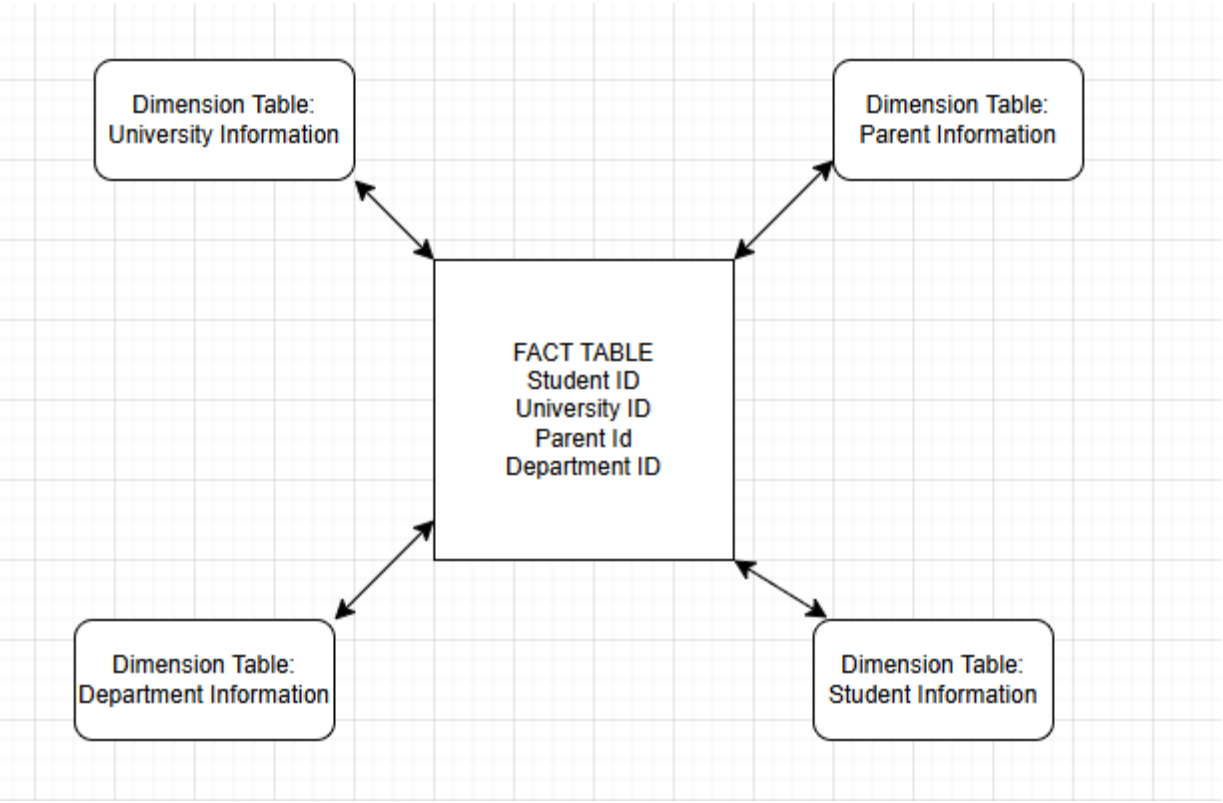
Question 2:

- Draw and explain a Star Schema.

Solution:

A star schema is a kind of schema where database relationships are stored in such a way that there is 1 fact table, which holds the numeric factual data that happened in real-time, and various other tables called dimension tables, which hold further information about the factual data, connect to that fact table. This layout, once drawn, makes it look like a star shaped schema.

The benefit of making the relationships follow star schema, is that if we are dealing with much larger databases, star schema makes the queries more optimized and faster to execute, as fewer joins are needed. We can also analyze and aggregate our data much easier in a star schema as a Data Engineer.



Reference: https://www.databricks.com/glossary/star-schema?

Question 3:

- **Compare ETL vs ELT in a table (pros/cons, tools, when to use).**

Solution:

ETL	ELT
ETL means: Extract, Transform, Load Data is extracted first, then it's cleaned and transformed, then finally loaded in the database.	ELT: Extract, Load, Transform Data is extracted first, then loaded in the database, and finally transformed or cleaned.
PROS OF ETL	**PROS OF ELT**
1. Data can be ready for use, as the moment it lands in database, it's already cleaned and transformed 2. If there are concerns about privacy, we can transform the data to hide some important data, before storing it in the database.	1. Data is loaded very fast because no transformation is done. 2. If we are not sure about transformation, we can load it now, then transform later according to our requirements
CONS OF ETL	**CONS OF ELT**
1. Time consuming if data needs to be used asap, as data needs to be transformed first, then loaded. 2. Data pipelines could need more work to update if someone changes the format in the data source.	1. Privacy concerns can arise if the data arrived has some sensitive information, so we would need to transform it later. 2. Because the data is not transformed, we would need a powerful database that can handle transformations with large volumes of data.
Tools for ETL	**Tools for ELT**
Informatica, Talend, Azure data factory, Google Cloud Dataflow, Fivetran and many more	Snowflake, BigQuery, Redshift, Spark
When to use?	**When to use?**
If we have an older system, or if the system is present on premises, we can use ETL when we need cleaned data, before storing it in some database. For example, suppose we are a multiplayer game studio, we might need to maintain gamer user's privacy, before storing their details in our database, so we could transform their sensitive data then store it in database.	If we are dealing with fast changing and large numbers of data, so much so that there is not enough time or need to transform, we could use ELT in those cases, then later transform data according to our usecases.

Reference: https://www.ibm.com/think/topics/elt-vs-etl

Question 4: Write 1 paragraph on Batch vs Streaming vs micro-batch with examples.

Solution:

When you load large amounts of data, at different intervals throughout the day, and later process it at the end of the day, it's called batch processing. For example, game developer might implement a leaderboard system for a game, that updates user ranking at the end of the day. Stream processing is when data is fetched and processed at the same time. For example, game developer might implement a system to quickly show the user his score after the end of his fortnite match. Micro-batch processing combines benefits of both batch processing and stream processing, so you benefit from both kinds of processing. For example, a game developer here might implement a system to take the user's rank data after every 5-10 minute interval, and update the dashboard for the user.

Reference: https://medium.com/%40raj.businT/batch-vs-micro-batch-vs-streaming-when-to-use-what-and-why-it-matters-f6063456c389

Question 5:

Explain Bronze/Silver/Gold layers with diagram.

Solution:

Bronze/Silver/Gold layers, also called a medallion architecture, is a data modelling technique that can be used by data engineers to improve data quality and business value for the end users.

The data flows from bronze layer, then to silver, and then gold layer. After every layer, the quality of the data improves, until it becomes business-ready data.

Bronze Layer:

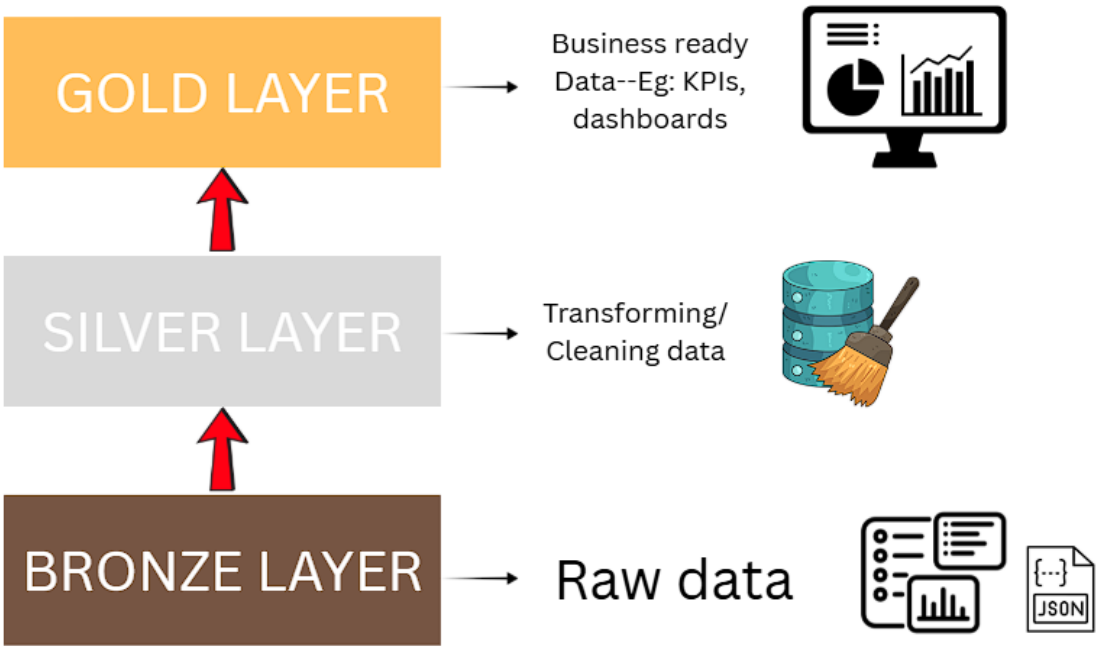
This is the raw data, that is fetched from various sources, such as APIs, LOGs, Streaming services etc. This data is uncleaned and non structured. Sometimes, we might get meta-data, but it's mostly unusable messy data. This layer holds all useless and useful data for future purposes.

Silver Layer:

This is where the big work is done. The data from the bronze layer, is finally cleaned, processed and transformed. We might remove duplicates, fix some missing values, enforce relationships between different data tables. This layer makes sure that the data is now reliable and good for analysis.

Gold Layer:

This layer holds the refined, ready to use data for the stake holders, who may now use this data for insights, analysis etc. The data in layer can further get aggregated, structured, and optimized for faster querying as well.



References:

- https://docs.databricks.com/gcp/en/lakehouse/medallion
- https://towardsdatascience.com/the-fundamentals-of-data-warehouse-data-lake-lake-house-ff640851c832/

Question 6:

Document 3 ingestion methods with pros/cons.

Solution:

Three types of ingestion methods exist.

1.Batch ingestion:

Load data first, then later process it.

Pros:

1. Very efficient for large datasets
2. Easy to manage
3. Cheaper

Cons:

1. Data is not readily available for use
2. Not useful enough for real-time needs if we need to urgently use data

Example: A gaming company might process player activity after the end of the day, to update their leaderboard.

2.Streaming ingestion:

Streaming ingestion is when data is processed, as it's arriving, allowing for real-time analysis.

Pros:

1. Faster access to data
2. Immediate usage of data
3. Can be scaled to handle large volumes of data

Cons:

1. Expensive setup
2. Complex management

Example: A gaming company, might monitor player actions in real time to check if they are hacking or not

3.Micro ingestion:

Combines the best of both worlds, of batch and streaming ingestion techniques. Data arrives in small intervals, and is processed also, at some fixed short intervals.

Pros:

1. Almost closer to real-time processing, helping us in data analysis
2. Simpler to implement than streaming ingestion technique
3. More cost effective than streaming ingestion technique

Cons:

1. A little delay can be expected, as it's not 100% real time
2. A little bit costly and more complex to setup, as we need setup for both batch and stream processing.

Example: A game company might update player statistics every 2-3 hours

References:

1. https://estuary.dev/blog/batch-processing-vs-stream-processing
2. https://hazelcast.com/foundations/software-architecture/micro-batch-processing
3. https://medium.com/@raj.busint/batch-vs-micro-batch-vs-streaming-when-to-use-what-and-why-it-matters-f6063456c389

Question 7: List 4 data quality dimensions with examples.

Solution:

Data quality dimensions are some characteristics of data, that can help us measure how valuable and reliable the data is, so it can be used by data analysts, engineers or scientists alike. We will study these 4 dimensions down below in the table.

Dimension	Explanation	Example
1. Accuracy	This tells us how accurate the data is	For example, is the user's score in a game accurate?
2. Completeness	This tells, if the data is complete	Has the user entered his complete details in the form?
3. Timelessness	Is the data up-to-date? And available when needed?	Does the current leaderboard reflects user's actual game rank?
4. Consistency	Is the data consistent across all the databases and systems?	Is the result being shown for the user, accurately displayed, as stored in the actual database?

References:

https://www.precisely.com/blog/data-quality/data-quality-dimensions-measure/

Question 8:

Explain schema evolution with forward vs backward compatibility

Solution:

Schema evolution:

Scheme evolution refers to a process, where the schema of the database, keeps changing to accommodate the needs of data engineers, in such a way that the structure of the database remains intact.

Two kinds of schema evolution can happen:

Forward Compatibility	Backward Compatibility
This means that the new data being entered is easily understood by systems and applications which are following some older schema.	This means that old data that is produced with the older schema, can still be read by newer systems and applications.

References:

1. https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution

Question 9:

Compare file formats (CSV vs Parquet) in a table.

Solution:

Row-Based Format:

The data is stored row by row, every record is a singular row, stored together in files such as CSV or JSON

Columnar-Based Format:

The data is stored in columns. For example, all values with a similar attribute might belong to some one column. It's stored in a parquet or ORC file.

Feature	Row-Based Data	Column Based Data
Storage Format	Row by row	Column by column
Compression:	Not very efficient, data types are all mixed	More efficient as similar data types stored together
Performance for READ	Slower	Faster
Performance for WRITE	FASTER	SLOWER
USECASE	OLTP systems, small data	OLAP systems, big data

References:

https://medium.com/@aiiaor/which-data-file-format-to-use-csv-json-parquet-avro-orc-e7a9acaaa7df

Question 10:

partitioning with an example (by year, month

Solution:

Data partitioning:

This is a practice of dividing a large volume data, into smaller chunks called partitions, that can help us improve performance, scalability and management of the data, especially if we are dealing with big Data.

Partitioning by Year and Month.

When we try portioning our data by month or year, we basicallt organize data into some folders or directories to make sure that data is reflecting those time periods accurately. This is useful for time series data, like logs, transactions or event records, so it helps querying it faster and easily enable filters.

Example :A game company might store player activity data in a month or year directories.

References:

https://www.pingcap.com/article/sql-partition-demystified-from-concept-to-implementation

Question 11:

Research one orchestration tool (Airflow/Prefect/Dagster) and describe.

Solution:

Apache Airflow:

Apache Airflow is a tool that helps us organize, schedule, and monitor workflows. A workflow is a series of steps that get executed to perform one task to another in a data engineering workflow. It could be like moving data from place to place, cleaning and transforming data, storing it in some database, or just monitoring the overall system, we can get all of this automated by defining some workflow in Apache Airflow.

We write the instructions for tasks using Python code. We can also define the order, to make sure what needs to run first, what can and cannot run in parallel, what needs to done only after something is done etc.

Components of Airflow

1. DAGs(Directed Acyclic Graphs)
   Airflow follows a DAGs style, which means directed acyclic graphs. Tasks are only run in 1 order, instead of a cycle. No loop is used. Tasks can happen 1 time, or in a schedule like daily, hourly, weekly, but they happen from one task to another in a workflow.
   For example : Source Data -→ Clean it → Store it → Monitor for another upcoming task.

2. Tasks and Operators
   A task is some step in the workflow.
   An operator contains the instructions for the task, like a blueprint. An operator can be something like a python script, sql query, sourcing data instructions etc.

3. Scheduler
   A scheduler is a component in airflow that can help us perform a workflow at some specific time or when certain conditions are met. We can schedule the workflows weekly, daily, weekly or when some certain conditions are met. We can use sensors that sense if a condition is met or not, then the scheduler can work for that task.
4. Executer and Workers

These components are the ones that perform the actual work. They execute the instructions defined in the workflows. They might perform these tasks inside our local machine or on the Cloud for scalability.

5. Metadata Database
   Inside airflow, metadata database, keeps track of every single component in the database, how it performed, when it performed, if somethings failed etc.
6. Web Interface
   This is just a dashboard or a UI, where we see everything at one place. These include our workflows that we have defined, their progress, logs, or options to retry the tasks that failed.

Usecase for Data Engineers

On the surface, it may look like airflow is just some random scheduler, but it's a very important tool that can be used by data engineers to automate their complex workflows.

A data engineer takes raw data from multiple sources, cleans and transforms it, then stores it at some place, and at the same time optimizes it. Instead of keep doing one thing again and again, data engineer can just write all these steps that repeat inside a workflow in airflow, and schedule it to happen it again and again, giving him more time to focus on other important tasks in his data engineering domain.

Some domains where airflow can help data engineers include

1. Automation: Automate tasks on a schedule, instead of running steps manually.
2. Scalability: Airflow can handle scalability, by distributing workload across the local machine or the cloud.
3. Reliability: Airflow alerts us if something failed, so it's reliable.
4. Clarity: Tasks can easily be defined and shown the user improving clarity.
5. Flexibility: It uses python, which is a very easy language to work with.

References:

https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html

Question 12:

Write 1 page on Distributed Computing (horizontal vs vertical scaling).

Solution:

Distributed Computing refers to the process where you distribute workload across multiple computers/servers/nodes, to reduce load and improve task efficiency. Sometimes, a single

system might get overwhelmed with so many tasks, then scaling or improving the current system would be the only way to handle more tasks and data, to avoid service disruption.

To grow performance or capacity, we use two ways for distributed computing.

Vertical Scaling

Vertical scaling refers to making the current system stronger. To make the current system efficient, you can add more RAM, replace with a better CPU, get faster, large and more efficient storage, or buy a motherboard with better PCI bandwidth.

Example:
Suppose I work as a data engineer remotely, I may be creating ETL pipelines on my own system and deliver them to clients, but my own system may not be as much efficient to handle complex heavy tasks. I can vertically scale my system by adding more RAM, changing to a better processor, or buying more storage.
Following are some of the ways I could upgrade:
16 GB ->32GB RAM
Ryzen 1600x --→Ryzen 5700x
1TB Harddisk -→ 10 TB Harddisk

Advantages:

1. Very simple to setup
2. No need for complex management
3. Easier network and data management

Disadvantages:

1. There is always a limit to how further you can upgrade
2. If the current system fails, your entire work gets disrupted

Horizontal Scaling

In horizontal scaling, we add more machines to the system, so our workload is spread across multiple nodes.

For example: As a data engineer, I may move my current workload to the cloud, and horizontally scale to multiple nodes if current systems are not able to handle big Data efficiently.

Advantages:

1. More reliable, as even if one system fails, rest of the systems can take over the tasks

2. Efficient scalability, we can easily add more nodes.

Disadvantages:

1. More expensive and complex to setup than Vertical Scaling
2. Network failures/delays

Usecases for Data Engineers:

If the data engineer deals with smaller workloads, with fever datasets, vertical scaling might be fine for personal tasks or smaller systems, but as you begin dealing with more and more big data, a single system might not be able to efficiently handle large amounts of data, too many computations have to run concurrently which is not efficient on vertical scaled computer, so horizontal scaling can easily help distribute large numbers of data computations across several nodes. Tools like Hadoop, spark or distributed databases are used for horizontal scaling and distribute workload across several machines.

Reference:

1. https://www.cockroachlabs.com/blog/vertical-scaling-vs-horizontal-scaling
2. https://www.datacamp.com/blog/horizontal-vs-vertical-scaling

Question 13:

Document indexing & caching basics in databases.

Solution:

Document Indexing:

Document indexing is a process where you fetch files via their index, to find rows efficiently, faster, without scanning the entire database. The index stores some value from one or two columns in a sorted way, and points to where full rows are in the storage.

Types of indices:

1. B-Tree
   This kind of index is very common. It's also known as B+-tree. It's structure in such a way that keeps all the data balanced, so operations such as deletion, insertion, updation, lookups are efficient as possible as the data grows bigger.
2. Hash Index:
   These indexes match for exact queries, such as user = id. You search by equality in this kind of index. For example: Finding users by their IDs.

3. Full-Text Index:
   This kind of index is very useful when searching inside large textual data such as paragraphs/documents. In the background, inverted indexing is being done here, which means that first data is flipped, then within that data it ssearches for words, and for each word, it shows which documents contain that word.

   Example : Ali is a hardworker

   Inverted Index:

Word	Documents
Ali	1

Is	1
a	2
hard	1
worker	1

   When a specific word is searched, the database can quickly look at the inverted index, and it knows exactly which documents contain those words.

Caching Basics in Databases:

Caching refers to copies of frequently accessed data, stored in some fast storage such as in memory, so frequent requests are served quickly without fetching data from slower large database.

Two kinds of caches exist.

- memory caching which is RAM, is very fast.
- Local cache and shared cache: Local cache is specific to an application server, while shared cache is accessible by multiple servers.

Caching benefits for Data Engineers:

As data engineers, we often work with large databasis, where frequent access of data is quite common, such as multiple users and applications querying the data, so we would need to do it very quickly.

If we implement a good index system, data can be queried quickly, bad or missing indexes make simple queries very difficult when data grows.

Likewise, caching helps offload repeated reads so the database isn't overloaded, especially during peak hours.

Disadvantages:

1. Too many indexes can also slow down writes so we should be careful where to implement it.
2. Bad caching implementation can cause poor results and inconsistencies.

References:

1. https://upsun.com/blog/basics-of-database-indexing/
2. https://www.prisma.io/dataguide/managing-databases/introduction-database-caching
3. https://www.dragonflydb.io/guides/database-caching

Question 14:

Explain compression (Snappy, Gzip, etc.) in simple terms.

Solution:

Compression:

Compression means condensing a large thing into smaller one without losing details.

Data Compression:

Data Compression refers to condensing the large size of files or data streams into smaller ones, making it easier and faster to store, transmit and process. This is very important for data engineers who deal with large datasets, as it can help optimize our storage and enhance performance.

Compression Algorithms:

Several kinds of compression algorithms exist, such as snappy, gzip.

Snappy	Gzip
Speed: Designed for high speed compression/decompression. Can	Speed: Not as fast as snappy but it can compress very good

compress data at 250 mb/s and decompress at 500 mb/s, if we are using a 64 bit single core processor.	
Compression Ratio: It's not efficient enough in terms of compression ratio, but good balance between speed and compression.	Compression Ratio: Efficient compression ratio compared to snappy. Can compress very large files into small sizes, but at cost of speed.
Usecase: We use this if we prefer speed and compression at same time.	Usecase: We use it where we prefer archiving large data, in scenarios where reducing storage spaceis more important than compression speed
Commonly used in systems such as Aparche Parquet and Kafka	

In the real world as a data engineer, if we are dealing with real-time data processing, we may go for snappy.

If we are dealing with limited space, and need long-term storage, we may go for GZip.

Question 15:

Research one data observability tool and document its purpose.

Solution:

Data observability tool is a software that can help data engineers monitor the health of their data systems. It provides us with important insights into the overall ETL pipeline, data quality and performance of the system, which can help us detect and resolve issues efficiently.

Purpose of Data Observability Tool

The purpose of data observatibility tool is to make sure that data system are working efficiently and correctly.

Following are some of the ways they help data engineers.

1. Monitoring:

They help us monitor data pipelines, to track flow of data between various stages, so we find out any potential failures.

2. Detecting Anomalies:

These help us find any unusual patterns in data, such as missing values or weird changes.

3. Root Cause Analysis:

These help us find root cause, that led to some problem.

4. Automated Alerts:

These can notify data engineers about any failures or issues t hat occur in the system, these help minimize damage and avoid downtimes.

5. Optimization:

These help us optimize performance of the data system, insuring reliability, security and speed.

6. Enhancing Data Quality for Efficiency:

The tools inside this software help improve our data's quality so while doing data analysis, data is accurate and reliable.

References:

https://atlan.com/data-observability

https://www.ibm.com/think/topics/data-observability

Question 16:

Compare APIs (REST vs GraphQL).

Solution:

APIs stand for application programming interface. They are a set of rules and protocols that allow different applications to communicate with each other. They act as a bridge between two applications, helping us request and exchange data seamlessly between them and show to the end user.

Two kinds of APIs we will study today:

REST API	GRAPHQL

Multiple endpoints: Each resources has its own URL. Like /systems for system data or /blogs for blog data.	Single Endpoint: All requests go to only for URL. For example : /graphql
Over-fetching: You may occasionally receive more data than is required. For example, even if you just need the user's name, retrieving them may also return their posts.	Client-Specified Queries: You specify the precise information you require from the server. For instance, you can ask for a user's name alone, without retrieving their posts.
Fixed Responses: What information is sent back is determined by the server. You may need to submit another request if you require more information.	Prevents Over-fetching and Under-fetching: You only receive the data you require because you specify it.
Caching: Because each endpoint is fixed, it is simpler to cache responses.	Real-Time Updates: Enables clients to receive updates whenever data changes by supporting real-time data subscriptions.

IMPORTANCE FOR DATA ENGINEERS:

1.By lowering the quantity of data transferred and the number of requests, GraphQL enables more effective data retrieval.

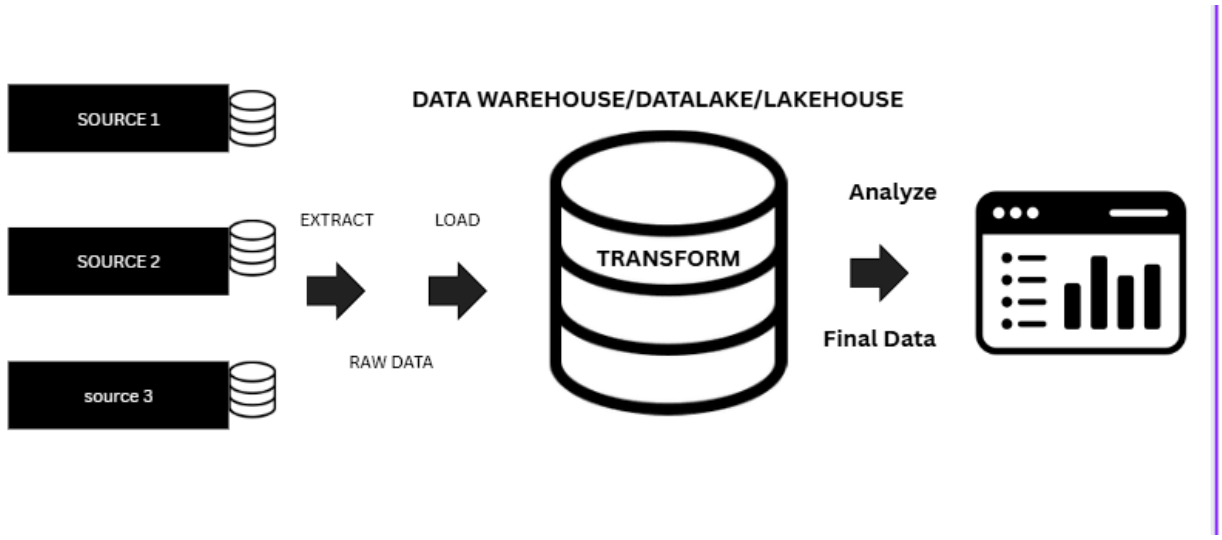
2. Data engineers can create APIs with GraphQL that meet particular client requirements, increasing the adaptability of data access.
3. REST and GraphQL are both scalable, but GraphQL's single endpoint and accurate data fetching make scaling strategies easier.

References

1. https://aws.amazon.com/compare/the-difference-between-graphql-and-rest
2. https://www.ibm.com/think/topics/graphql-vs-rest-api

Question 17:

Draw a mini diagram of a pipeline with ingestion → storage → transformation → consumption.



Question 18:

Create a glossary of 20 keywords from this week's syllabus.

Solution:

1. Data Warehouse

Data warehouses are centralized repositories of structured, cleaned data that are ready for analysis.

2. Data Lake

Data lakes are centralized repositories that hold unstructured, semi-structured, and structured data without a set schema.

3. Data Lakehouse

A hybrid storage solution that supports both structured and semi-structured data by fusing the capabilities of data lakes and data warehouses.

4. Star Schema

A database schema known as a "star schema" has a central fact table that is connected to several dimension tables.

5. ETL (Extract, Transform, Load)

Data is extracted, transformed, and then loaded into storage as part of the ETL data integration process.

6. ELT (Extract, Load, Transform)

Data integration that involves extracting, loading, and transforming data inside a database.

7. Batch Processing

Batch processing is the process of gathering data over time and processing it in large quantities at predetermined times.

8. Stream Processing

Real-time data processing as it comes in.

9. Micro-Batch Processing

This method handles brief, regular data intervals, combining batch and stream processing.

10. Bronze/Silver/Gold Layers

Medallion architecture layers for raw, cleaned, and business-ready data.

11. Data Ingestion

Importing data into a storage system through batch, streaming, or micro-batch techniques.

12. Data Quality Dimensions

Attributes used to gauge the dependability of data, such as accuracy, completeness, timeliness, and consistency.

13. Schema Evolution

The process of making changes to a database schema over time while preserving backward and forward compatibility.

14. Row-Based vs Columnar Storage

Columnar stores data column by column, optimizing read/write and compression; row-based stores data row by row.

15. Partitioning

The process of breaking up big datasets into manageable portions according to time or other characteristics to enhance performance.

16. Orchestration Tool (Airflow)

Software for scheduling, managing, and tracking workflows across tasks, ensuring automation and dependability.

17. Distributed Computing

Using several computers or nodes to share workloads, scalable either vertically or horizontally.

18. Indexing

The process of arranging data in databases (such as B-Tree, Hash, and Full-Text) to expedite retrieval operations.

19. Caching

The practice of keeping frequently accessed data in fast memory to speed up retrieval and lessen database load.

20. Data Observability Tool

Software to track, identify irregularities, and enhance data pipelines for performance and quality.

Question 19:

Document a messaging pattern (Kafka, RabbitMQ, Pub/Sub).

Solution:

A messaging pattern is how different systems talk to each other by sending messages instead of directly calling each other. Tools like Kafka, RabbitMQ, and Google Pub/Sub help move data reliably between apps. For example, in a game, when a player scores, a message can be sent through Kafka to update the leaderboard without slowing down the game itself. These patterns make systems scalable, reliable, and decoupled, meaning each part can work independently.

Reference: https://www.confluent.io/blog/event-driven-architecture-kafka-rabbitmq

Question 20 :Define Data Contracts & their importance.

Solution:

Data Contracts & Their Importance

A data contract is like a promise between teams or systems about the shape and rules of the data they exchange. It says, “This is what the data will look like, and this is how you can use it.” It’s important because it prevents errors, ensures compatibility, and keeps pipelines

running smoothly, especially when multiple teams or apps rely on the same data. For example, if one system sends user info in a fixed format, the receiving system knows exactly what to expect.

Reference: https://www.oreilly.com/library/view/data-ops/9781492056690/ch04.html